Data Science for Economists

Lecture 6: Misc Base R

Jiaxi Li
University of North Carolina | ECON 370

Table of contents

Introduction

Agenda

Today we will cover a bunch of miscellaneous topics that don't quite fit anywhere else.

I will also mention some functions and tips and tricks that I find useful.

Pipeline

We're in (New) Pipe-land Now

Now, there is a pipe `▷` (|>) even in basic R (no package needed)! It lets you send (i.e. "pipe") intermediate output to another command.

In other words, it allows us to chain together a sequence of simple operations and thereby implement a more complex operation.

I want to reiterate how cool pipes are, and how using them can dramatically improve the experience of reading and writing code. Compare:

```
## These next two lines of code do exactly the same thing.
mpg ▷ subset(manufacturer=="audi") ▷ aggregate(hwy ~ model, FUN = mean)
aggregate(subset(mpg, manufacturer=="audi"), hwy ~ model, FUN = mean)
```

The first line reads from left to right, exactly how I thought of the operations in my head.

- Take this object (`mpg`), do this (`filter`), then do this (`group_by`), etc.

The second line totally inverts this logical order (the final operation comes first!)

- Who wants to read things inside out?

An aside on pipes: |> (cont.)

The piped version of the code is even more readable if we write it over several lines. Here it is again and, this time, I'll run it for good measure so you can see the output:

```
mpg ▷
  subset(manufacturer=="audi") ▷
  aggregate(hwy ~ model, FUN = mean)
```

```
##         model      hwy
## 1          a4 28.28571
## 2 a4 quattro 25.75000
## 3 a6 quattro 24.00000
```

Remember: Using vertical space costs nothing and makes for much more readable/writeable code than cramming things horizontally.

PS — The pipe is originally from the magrittr package, which can do some other cool things if you're inclined to explore. It's been so popular that even SQL is adopting a pipe syntax. SQL!

It is a very powerful tool. People used to use `%>%` in **magrittr**. However, it is so popular that the base R starts to have it as `▷` after R version 4.1.0 in 2021!

Dates and Times

Dates and Time

In order to work with dates and times correctly, they need to be specified as a date or time.

You will likely work with dates more often than times, so that is where more attention will be placed.

Understanding exactly how dates work requires understanding a bit more about the advanced aspects of R; however, to get the basics does not require this.

The default format for dates in R is `YYYY-MM-DD.`

```
# Convert to date with base function
as.Date("2021-09-14")
```

```
## [1] "2021-09-14"
```

```
Sys.Date()
```

```
## [1] "2025-10-09"
```

If you have a date not in the format listed above, you have to give the `as.Date()` function the "format" argument. See this link for different formats.

Lubridate

While we have been working in base `R` almost exclusively so far, I would highly recommend using the `lubridate` package.

Working with dates and times is much easier than in base `R`.

```
library(lubridate)
```

```
# Convert to date using lubridate functions
ymd("2021-09-14")
```

```
## [1] "2021-09-14"
```

```
mdy("09/14/2021")
```

```
## [1] "2021-09-14"
```

```
today()
```

```
## [1] "2025-10-09"
```

Lubridate (Cont.)

```
# Extract a specific feature of the date
year(today())
```

```
## [1] 2025
```

```
month(today())
```

```
## [1] 10
```

```
week(today())
```

```
## [1] 41
```

```
# More detailed date
ymd_hms("2017-01-31 20:11:59")
```

```
## [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

```
## [1] "2017-01-31 08:01:00 UTC"
```

Date Comparison

It is easy to do comparison with dates. You can directly use logical comparisons `>` and `<`.

```
ymd("2021-09-14") > ymd("2021-08-12")
```

```
## [1] TRUE
```

```
ymd("2021-09-14") < "2021-08-12"
```

```
## [1] FALSE
```

```
# R can understand comparison between characters formatted as Date
"2021-09-14" > "2021-08-12"
```

```
## [1] TRUE
```

```
"2021-09-14" < "2021-08-12"
```

```
## [1] FALSE
```

This is very handy if you need to do logical indexing with dates!

More About Dates and Times

There are a lot of little quirks when it comes to dates and times that usually you will not have to know. God bless you if you're ever working with time zones, spring forward, etc.

This is where the skill of Googling/LLM really comes in handy.

I would recommend also reading Chapter 16 in *R for Data Science* that is listed on the syllabus.

Read/Write Files

Reading in Files

Because we've been working with simulated data or preloaded data, we have yet to have to read in data. There are four different types of files we will talk about today:

- csv: Work for all programs. Plain text (human-readable). Really close to Excel. Slow to write/read and large size.
- rds: Originated in R. There are special packages to load it into Python and Julia. Fast to write/read and small size.
- feather: Also works for Python and Julia. Very fast to write/read and medium size.
- fst: Originated in R. There are special packages to load it into Python and Julia. Ultra-fast to read/write even for massive datasets; small–medium (adjustable compression).

CSV

The most common function you will use is `read.csv()`.

- While you might want to read in Excel spreadsheets, I would highly recommend minimizing how much you use Excel for R.

To read in a file, you need to know where the file is located. You will need to set your "working directory."

```
# Get current working directory
getwd()
```

```
## [1] "D:/R/ECON370/lec6"
```

```
# Set a new working directory
setwd("D:/R/ECON370/lec6/")
getwd()
```

```
## [1] "D:/R/ECON370/lec6"
```

See Files in Directory

To see the names of the files in a certain directory, use the `list.files()` functions.

```
# list all files/folders in the current working directory
list.files()
```

```
##  [1] "06-miscR.html"                  "06-miscR.pdf"
##  [3] "06-miscR.Rmd"                   "blp_data.csv"
##  [5] "blp_data.rds"                   "blp_data2.csv"
##  [7] "blp_data2.rds"                  "Lec6_2024"
##  [9] "Lecture6Markdown_complete.Rmd" "Lecture6Script.R"
## [11] "libs"                           "OTC_data.feather"
## [13] "OTC_data.fst"
```

```
# list all files/folders in the specified working directory
# list.files("D:/R/ECON370")
list.files("D:/R/ECON370/HW1")
```

```
## [1] "2024"            "Assignment1.pdf" "Assignment1.Rmd" "marsh"
```

Read in CSV

To read in the CSV listed above (about market share data and prices), use the function `read.csv()`:

```
# OTC_data = read.csv("https://github.com/JiaxiLi1995/ECON370/raw/refs/heads/main/lec
# write.csv(OTC_data, file = "blp_data.csv", row.names = F)
OTC_data = read.csv("blp_data.csv")
```

We can save the csv using the function `write.csv()`:

```
write.csv(OTC_data, file = "blp_data2.csv", row.names = F)
```

A Few Things on CSV

CSV stands for "comma-separated values" i.e. the data values are separated by commas.

If you notice in the help documentation for read.csv, it assumes `sep=","`.

If this is not true, you might have to change the sep argument.

- e.g. Tab separated values are also common.

However, don't worry too much about this. While you need to understand it a little, we will be using `read_csv()` from the `tidyverse` and/or `fread()` from the `data.table` package, which are superior to read.csv.

rds

The rds files are native to R. We can try to save as rds file and read it with `saveRDS()` and `read.rds()`:

```
saveRDS(OTC_data, file = "blp_data.rds")
OTC_data_rds = readRDS("blp_data.rds")
```

You can also compress the file when saving it:

```
saveRDS(OTC_data_rds, file = "blp_data2.rds", compress = T)
```

feather

Feather files are designed for fast, efficient, and easy data interchange between languages. `.feather` is usually the file extension. We will use a special package, `arrow`, to save (`write_feather()`) and load (`read_feather()`) feather file.

```
library(arrow)
# Feather
write_feather(OTC_data, "OTC_data.feather", compression = T)

OTC_data_feather = read_feather("OTC_data.feather")
```

fst

We can also work with `.fst` file in R. We will use `arrow` package to save (`write_fst()`) and load (`read_fst()`) fst file.

```
library(fst)
write_fst(OTC_data, "OTC_data.fst")

OTC_data_fst = read_fst("OTC_data.fst")
```

Speed Comparison

```
## Unit: microseconds
##          expr    min     lq    mean  median     uq    max neval
##      rds_read  384.3  402.8  433.19  423.60  466.8  506.3    10
##      csv_read 2993.7 3117.9 3288.98 3189.30 3354.3 4151.7    10
##  feather_read 2880.8 2938.6 3208.39 3218.55 3392.4 3583.0    10
##      fst_read  275.5  289.2  340.18  337.95  400.5  402.3    10

## Unit: microseconds
##          expr     min      lq     mean   median      uq     max neval
##      rds_save  1693.8  1718.4  1789.55  1771.25  1788.5  1965.1    10
##      csv_save 12990.8 13058.7 13184.75 13109.95 13329.1 13599.0    10
##  feather_save  2492.3  2555.8  2859.19  2783.25  2839.7  4059.5    10
##      fst_save   416.9   504.8   661.87   542.95   604.6  1520.9    10
```

Webscrapping

Internet as Data

1. Server-side
    - Data stored at the server, which sends HTML code with data in it to us
    - **Our process**: trudging through CSS selectors
2. Client-side
    - Our browser requests data from the server, server sends specific info we asked for
    - **Our process**: pinging an API endpoint

We will only demonstrate one example with `rvest`, which is server-side scrapping. There are more resources on webscrapping:

- Grant R. McDermott's lecture on webscrapping section 1 and section 2
- rvest

We will use `rvest`, `dplyr` and `janitor` here (`lubridate` is already loaded):

```
library(rvest)
library(dplyr)
library(janitor)
```

Before we get started

1. Be a good internet user
2. It's easy to accidentally kill some poor website
3. It's probably legal?

Some website puts more limit to webscrapping (such as X, formally known as Twitter).

See the warnings from the writer of `rvest`.

If you request a lot of data with scrapping, be polite with something like `Sys.sleep(2)` in between.

Use Rscript (`.R`) file to scrap data **only once** and save the data to some file (such as `.rds`). Then, read in the data file for your project. **Try not to scrap over and over again in your `.Rmd` file!**

HTML Basics

Here's some simple HTML:

```
<html>
<head>
  <title>Page title</title>
</head>
<body>
  <h1 id='first'>A heading</h1>
  <p>Some text &amp;amp; <b>some bold text.</b></p>
  <img src='myimg.png' width='100' height='100'>
</body>
```

HTML Basics

Here's some simple HTML:

```
<html>
<head> ## head is an element #<<
  <title>Page title</title>
</head>
<body>
  <h1 id='first'>A heading</h1>
  <p>Some text &amp;amp; <b>some bold text.</b></p>
  <img src='myimg.png' width='100' height='100'>
</body>
```

HTML Basics

Here's some simple HTML:

```
<html>
<head>
  <title>Page title</title>
</head>
<body> ## as is body #<<
  <h1 id='first'>A heading</h1>
  <p>Some text &amp;amp; <b>some bold text.</b></p>
  <img src='myimg.png' width='100' height='100'>
</body>
```

HTML Basics

Here's some simple HTML:

```
<html>
<head>
  <title>Page title</title>
</head>
<body>
  <h1 id='first'>A heading</h1> ## id='first' is an attribute #<<
  <p>Some text &amp;amp; <b>some bold text.</b></p>
  <img src='myimg.png' width='100' height='100'>
</body>
```

HTML Basics

Here's some simple HTML:

```
<html>
<head>
  <title>Page title</title>
</head>
<body>
  <h1 id='first'>A heading</h1>
  <p>Some text &amp;amp; <b>some bold text.</b></p> ## stuff in between is contents #<<
  <img src='myimg.png' width='100' height='100'>
</body>
```

Lucky for us: we don't need to write HTML. Just read it.

HTML Basics - Tags

- Tags all start with `<tag>` and end with `</tag>`
- Every page is in an `<html>` element with 2 children:
  - `<head>` contains metadata
  - `<body>` contains content you see
- Block tags form the overall structure of a page
  - `<h1>` provides a heading
  - `<p>` is a paragraph
- Inline tags like `<b>` (bold), `<i>` (italics), and `<a>` (links) exist
- Just a sample of tags, can look up others you don't know
- The rest is the content

Example: Scraping Wikipedia

Let's imagine we want to scrape the Men's 100 metres world record progression page on Wikipedia.

In particular, we want to get the information from the 3 main tables on the webpage.

Here's what happens when we give R no instructions at all:

```
raw_wiki ← read_html("https://en.wikipedia.org/wiki/Men%27s_100_metres_world_record_p
raw_wiki
```

```
## {html_document}
## <html class="client-nojs vector-feature-language-in-header-enabled vector-feature-langua
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
## [2] <body class="skin--responsive skin-vector skin-vector-search-vue mediawik ...
```

```
class(raw_wiki)
```

```
## [1] "xml_document" "xml_node"
```

Parsing HTML: Inspecting Web Pages

We can use `inspect` to get a better sense of what is available on a given webpage. What tags can we use to grab the desired information from our wikipedia page?

It looks like we want to grab information from tables with the class "wikitable."

Brief Aside: CSS Selectors

We can parse HTML using **CSS Selectors**, which define patterns for locating HTML elements.

CSS Selectors are pretty complex, and we're going to keep it light today. If you want to learn more, check out the CSS Diner. Additionally, if CSS Selectors are giving you tons of trouble, check out SelectorGadget.

4 selectors to know:

1. `p` selects all `<p>` elements
2. `.title` selects all elements with `class` "title"
3. `p.special` selects all `<p>` elements with `class` "special"
4. `#title` selects the unique element with id attribute that equals "title"

Getting Our Tables

We can use the selector `table` to select all `<table>` elements:

```
raw_wiki |> html_elements("table")
```

```
## {xml_nodeset (15)}
##  [1] <table class="box-Unreferenced_section plainlinks metadata ambox ambox-c ...
##  [2] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Athlete\n</ ...
##  [3] <table class="wikitable" style="text-align: left;"><tbody>\n<tr>\n<td st ...
##  [4] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Wind\n</th> ...
##  [5] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Wind\n</th> ...
##  [6] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Athlete\n</ ...
##  [7] <table class="nowraplinks mw-collapsible autocollapse navbox-inner" styl ...
##  [8] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbo ...
##  [9] <table class="nowraplinks hlist mw-collapsible autocollapse navbox-inner ...
## [10] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbo ...
## [11] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbo ...
## [12] <table class="nowraplinks mw-collapsible uncollapsed navbox-inner" style ...
## [13] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbo ...
## [14] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbo ...
## [15] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbo ...
```

Alas, so many other things.

Trying again

We want tables with class wikitable: `table.wikitable` should work!

```
raw_wiki ▷ html_elements("table.wikitable")
```

```
## {xml_nodeset (5)}
## [1] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Athlete\n</t ...
## [2] <table class="wikitable" style="text-align: left;"><tbody>\n<tr>\n<td sty ...
## [3] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Wind\n</th>\ ...
## [4] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Wind\n</th>\ ...
## [5] <table class="wikitable"><tbody>\n<tr>\n<th>Time\n</th>\n<th>Athlete\n</t ...
```

Looks better!

Accessing the actual info

We still don't have the table info. `html_table()` can help:

```
table_dfs ← raw_wiki ▷ html_elements("table.wikitable") ▷
  html_table()
table_dfs[[1]]
```

```
## # A tibble: 21 × 5
##     Time Athlete                Nationality    `Location of races`     Date
##    <dbl> <chr>                  <chr>          <chr>                   <chr>
##  1  10.8 Luther Cary            United States  Paris, France           July 4, 1…
##  2  10.8 Cecil Lee              United Kingdom Brussels, Belgium       September…
##  3  10.8 Étienne De Ré          Belgium        Brussels, Belgium       August 4,…
##  4  10.8 L. Atcherley           United Kingdom Frankfurt/Main, Germany April 13,…
##  5  10.8 Harry Beaton           United Kingdom Rotterdam, Netherlands  August 28…
##  6  10.8 Harald Anderson-Arbin  Sweden         Helsingborg, Sweden     August 9,…
##  7  10.8 Isaac Westergren       Sweden         Gävle, Sweden           September…
##  8  10.8 Isaac Westergren       Sweden         Gävle, Sweden           September…
##  9  10.8 Frank Jarvis           United States  Paris, France           July 14, …
## 10  10.8 Walter Tewksbury       United States  Paris, France           July 14, …
## # ℹ 11 more rows
```

...this is crazy! `R` is cool sometimes.

General Workflow

Your workflow using `rvest`: get html --> get desired html elements --> break into individual elements --> turn into table/text/numbers/whatever.

Useful commands for getting individual elements:

- `html_text2()` gets plain text contents of an HTML element
- `html_attr()` gets attributes from an HTML element (e.g., links)
- `html_table()` creates a data.frame from a table in an HTML element

A Little Cleanup

Let's get our data frames in working order here:

```
table_dfs_int ← raw_wiki ▷ html_elements("table.wikitable") ▷
  html_table()

table_dfs ← lapply(table_dfs_int[c(1,3,4)], # drop unwanted tables
                   function(x) x ▷
                     clean_names() ▷ ## fix colnames, from the janitor package
                     mutate(date = mdy(date))) ## from lubridate
table_dfs[[1]]
```

```
## # A tibble: 21 × 5
##     time athlete              nationality    location_of_races       date
##    <dbl> <chr>                <chr>          <chr>                   <date>
##  1  10.8 Luther Cary          United States  Paris, France           1891-07-04
##  2  10.8 Cecil Lee            United Kingdom Brussels, Belgium       1892-09-25
##  3  10.8 Étienne De Ré        Belgium        Brussels, Belgium       1893-08-04
##  4  10.8 L. Atcherley         United Kingdom Frankfurt/Main, Germany 1895-04-13
##  5  10.8 Harry Beaton         United Kingdom Rotterdam, Netherlands  1895-08-28
##  6  10.8 Harald Anderson-Arbin Sweden        Helsingborg, Sweden     1896-08-09
##  7  10.8 Isaac Westergren     Sweden         Gävle, Sweden           1898-09-11
##  8  10.8 Isaac Westergren     Sweden         Gävle, Sweden           1899-09-10
##  9  10.8 Frank Jarvis         United States  Paris, France           1900-07-14
## 10  10.8 Walter Tewksbury     United States  Paris, France           1900-07-14
```

Combined Data

```
wr100 ← rbind(
  table_dfs[[1]] ▷ select(time, athlete, nationality, date) ▷
    mutate(era="Pre-IAAF"),
  table_dfs[[2]] ▷ select(time, athlete, nationality, date) ▷
    mutate(era="Pre-automatic"),
  table_dfs[[3]] ▷ select(time, athlete, nationality, date) ▷
    mutate(era="Modern")
)
head(wr100)
```

```
## # A tibble: 6 × 5
##    time athlete               nationality    date       era
##   <dbl> <chr>                 <chr>          <date>     <chr>
## 1  10.8 Luther Cary           United States  1891-07-04 Pre-IAAF
## 2  10.8 Cecil Lee             United Kingdom 1892-09-25 Pre-IAAF
## 3  10.8 Étienne De Ré         Belgium        1893-08-04 Pre-IAAF
## 4  10.8 L. Atcherley          United Kingdom 1895-04-13 Pre-IAAF
## 5  10.8 Harry Beaton          United Kingdom 1895-08-28 Pre-IAAF
## 6  10.8 Harald Anderson-Arbin Sweden         1896-08-09 Pre-IAAF
```

Presidential Speech and Fed

There are many textual data online are publicly available.

If you are interested, you can scrap US presidential speech from The American Presidency Project (which is the most complete archive) using `rvest`.

You can enter "Webscrapping presidential speech from the American Presidency Project (which is the most complete archive) using rvest (R)" in ChatGPT to get the structured code.

You can also scrap the Federal Reserve Annoucements.

Regular Expressions

Parsing Text

Parsing text to find patterns or extract data is hard.

There are a group of functions that can be used for this: the `grep()` functions.

`grep()` takes a pattern and character vector and returns a vector of indexes for which elements contained the pattern.

`grepl()` takes a pattern and character vector and returns a logical vector for if each element contains the pattern.

`sub()` takes a pattern, a replacement, and a character vector and returns the character vector with the first occurrence of the pattern replaced with the replacement.

`gsub()` is exactly like `sub()` except that it replaces all occurrences.

Parsing Text: Examples

```
statecountry_names = c('Florida','Germany','Georgia','Geniva',
                'Istanbul','NewZealand','Australia')
grep("G",statecountry_names)
```

```
## [1] 2 3 4
```

```
grepl("G",statecountry_names)
```

```
## [1] FALSE  TRUE  TRUE  TRUE FALSE FALSE FALSE
```

```
grepl("A",statecountry_names,ignore.case = F)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE  TRUE
```

```
grepl("A",statecountry_names,ignore.case = T)
```

```
## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

Parsing Text: Examples

```
sub("G","A",statecountry_names)
```

```
## [1] "Florida"    "Aermany"    "Aeorgia"    "Aeniva"     "Istanbul"
## [6] "NewZealand" "Australia"
```

```
sub("G","A",statecountry_names,ignore.case = T)
```

```
## [1] "Florida"    "Aermany"    "Aeorgia"    "Aeniva"     "Istanbul"
## [6] "NewZealand" "Australia"
```

```
gsub("G","A",statecountry_names,ignore.case = T)
```

```
## [1] "Florida"    "Aermany"    "AeorAia"    "Aeniva"     "Istanbul"
## [6] "NewZealand" "Australia"
```

Regular Expressions

Sometimes there are patterns that you would like to detect in text that are more complicated than just a short character expression that you can program.

Or you'd like to do all combinations of something.

This is where regular expressions can be used.

Truthfully, we could spend an entire class just learning regular expressions.

As such, I am only going to briefly touch them and encourage you to look into them more on your own.

The plus side is they are pretty universal across languages. That is, whether you're using `R` or `Python`, regular expressions still work the same way!

Regular Expressions: grepl Examples

Let's say we want to flag character strings that start with "The". The regular expression would be `^The`

```
test_char = c("The beginning.","The end.","The middle.","Other.")
grepl("^The",test_char)
```

```
## [1]  TRUE  TRUE  TRUE FALSE
```

Now, let's say we want to flag character strings that end with "end." The regular expression would be `end.$`

```
grepl("end.$",test_char)
```

```
## [1] FALSE  TRUE FALSE FALSE
```

Find all vowels

```
grepl("[aeiou]",letters)
```

```
##  [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [13] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [25] FALSE FALSE
```

Regular Expressions: More grepl

Let's say we have a couple strings to match:

```
test_char = c("The beginning.","The end.","The middle.","Other.")
grepl("beginning|end",test_char)
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

We can also grab numbers from a string:

```
grepl("[0-9]",c(1:5,"I had 2 coffees today","Hello World!"))
```

```
## [1]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE
```

Regular Expressions: Only csv

We have talked about how to use `list.files()` to get files names in a folder:

```
# list all files/folders in the specified working directory
list.files("D:/R/ECON370/lec6")
```

```
##  [1] "06-miscR.html"                    "06-miscR.pdf"
##  [3] "06-miscR.Rmd"                     "blp_data.csv"
##  [5] "blp_data.rds"                     "blp_data2.csv"
##  [7] "blp_data2.rds"                    "Lec6_2024"
##  [9] "Lecture6Markdown_complete.Rmd" "Lecture6Script.R"
## [11] "libs"                             "OTC_data.feather"
## [13] "OTC_data.fst"
```

Suppose that I only want to have all the files with `.csv` extension (could be multiple files), we can use regular expressions:

```
list.files(
  path = "D:/R/ECON370/lec6",
  pattern = ".*\\.csv$",
  full.names = TRUE
)
```

```
## [1] "D:/R/ECON370/lec6/blp_data.csv"  "D:/R/ECON370/lec6/blp_data2.csv"
```

Regular Expressions: More

Validating that a user supplied an email to a website form is famously a difficult problem. Here's one regex that a user on StackOverflow put forth:

```
pattern <- (?:[a-z0-9!#$%&'+/=?^_`{|}~-]+(?:.[a-z0-9!#$%&'+/=?^_`{|}~-]+)|'(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\[\x01-\x09\x0b\x0c\x0e-\x7f])')@(?:(?:a-z0-9?.)+a-z0-9?|[(?:(?:(2(5[0-5]|[0-4][0-9])|1[0-9][0-9]|[1-9]?[0-9])).){3}(?:(2(5[0-5]|[0-4][0-9])|1[0-9][0-9]|[1-9]?[0-9])|[a-z0-9-]*[a-z0-9]:(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\[\x01-\x09\x0b\x0c\x0e-\x7f])+)])
```

Regular Expressions: More

Regular expressions are very powerful for processing text data. Processing text data is a skill set in-and-of itself.

At one point, I knew regular expressions (regex) decently well. Now, I have LLMs write my regexs if they're at all complicated.

Lots of learning when it comes to programming is simply being aware of something so that you can look into it more when you actually need to use it.

If you want to learn more about regular expressions, see the following cheat-sheet.

Functions That I Use

Misc Functions

We have seen:

- `rep()`: repeats a vector N times
  - Used a lot for preallocating vectors
- `sum()`: sums up vectors
- `seq()`: creates sequences
- `cbind()`: combine matrices or data.frames by columns
- `rbind()`: combine matrices or data.frames by rows
- `ncol()`: returns number of columns of data.frame or matrix
- `nrow()`: returns number of rows of data.frame or matrix
- `dim()`: returns dimensions of an array-like object

New Ones:

- `unique()`: takes the unique values of a vector
- `sort()`: sorts a vector
- `order()`: like sort, but gives the IDs rather than a sorted vector
- `substr()`: returns a string subsetted by the indexes supplied
- `strsplit()`: splits a string by a pattern

Examples

```
rep(0,5)
```

```
## [1] 0 0 0 0 0
```

```
rep(1:10,2)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10  1  2  3  4  5  6  7  8  9 10
```

```
rep(1:10,each=2)
```

```
##  [1]  1  1  2  2  3  3  4  4  5  5  6  6  7  7  8  8  9  9 10 10
```

```
letter_samp = sample(letters,12,replace=T)
letter_samp
```

```
##  [1] "n" "y" "w" "c" "h" "p" "l" "y" "n" "c" "n" "g"
```

```
unique(letter_samp)
```

```
## [1] "n" "y" "w" "c" "h" "p" "l" "g"
```

Examples (Cont.)

```
norm_draws = rnorm(10)
norm_draws
```

```
##  [1] -0.7729782  2.1499193 -1.3343536  0.4958705  1.2339762  0.6343621
##  [7]  0.4120223  0.7935853 -0.1524106 -0.2288958
```

```
sort(norm_draws)
```

```
##  [1] -1.3343536 -0.7729782 -0.2288958 -0.1524106  0.4120223  0.4958705
##  [7]  0.6343621  0.7935853  1.2339762  2.1499193
```

```
order(norm_draws)
```

```
##  [1]  3  1 10  9  7  4  6  8  5  2
```

```
sum(norm_draws)
```

```
## [1] 3.231097
```

```
seq(1,2.4,by=0.1)
```

```
##  [1] 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0 2.1 2.2 2.3 2.4
```

Examples (Cont.)

```
rand_mat1 = matrix(rnorm(2*3),nrow=3)
rand_mat2 = matrix(rnorm(1*3),nrow=3)
rand_mat1
```

```
##            [,1]         [,2]
## [1,] -0.9007918  0.619283535
## [2,] -0.7350262 -0.006198262
## [3,] -1.4276858 -0.685706846
```

```
rand_mat2
```

```
##            [,1]
## [1,] -0.2793335
## [2,] -0.7827303
## [3,] -0.7789972
```

```
cbind(rand_mat1,rand_mat2)
```

```
##            [,1]         [,2]       [,3]
## [1,] -0.9007918  0.619283535 -0.2793335
## [2,] -0.7350262 -0.006198262 -0.7827303
## [3,] -1.4276858 -0.685706846 -0.7789972
```

Examples (Cont.)

```
rand_mat1 = matrix(rand_mat1,nrow=2)
rand_mat2 = matrix(rand_mat2,nrow=1)
rand_mat1
```

```
##            [,1]       [,2]         [,3]
## [1,] -0.9007918 -1.4276858 -0.006198262
## [2,] -0.7350262  0.6192835 -0.685706846
```

```
rand_mat2
```

```
##            [,1]       [,2]       [,3]
## [1,] -0.2793335 -0.7827303 -0.7789972
```

```
rbind(rand_mat1,rand_mat2)
```

```
##            [,1]       [,2]         [,3]
## [1,] -0.9007918 -1.4276858 -0.006198262
## [2,] -0.7350262  0.6192835 -0.685706846
## [3,] -0.2793335 -0.7827303 -0.778997240
```

Examples (Cont.)

```
ncol(rand_mat2)
```

```
## [1] 3
```

```
nrow(rand_mat2)
```

```
## [1] 1
```

```
dim(rand_mat2)
```

```
## [1] 1 3
```

```
my_string = "Hello, This is Econ 370, and my name is Jiaxi"
substr(my_string,1,10)
```

```
## [1] "Hello, Thi"
```

```
strsplit(my_string,",")
```

```
## [[1]]
## [1] "Hello"                " This is Econ 370"     " and my name is Jiaxi"
```

Next lecture: Tidyverse